



ATENÇÃO: Esta atividade **não vale nota e não contará para os 10 pontos de atividades**. Ela foi disponibilizada apenas como **material de revisão e estudo para a prova**.

1. Implemente uma classe `Fracao` em Java, aplicando os conceitos de orientação a objetos estudados em sala. Sua classe deve possuir atributos privados para **numerador** e **denominador**, construtores adequados, métodos de acesso e métodos para realizar operações entre frações.

A classe deve possuir, no mínimo:

- construtor padrão, que inicialize a fração como 0/1;
- construtor parametrizado, que receba numerador e denominador;
- método `setFracao(int numerador, int denominador)` para definir os valores da fração;
- métodos `getNumerador()` e `getDenominador()`;
- método `imprimirFracao()` para exibir a fração no formato a/b;
- método `imprimirDecimal()` para exibir o valor decimal da fração;
- método `simplificar()` para reduzir a fração à forma simplificada;
- método `somar(Fracao f)`;
- método `subtrair(Fracao f)`;
- método `multiplicar(Fracao f)`;
- método `dividir(Fracao f)`.

Considere ainda as seguintes regras:

- o denominador não pode ser igual a zero;
- os atributos da classe devem ser privados, garantindo o encapsulamento;
- sempre que necessário, a fração deve ser apresentada de forma simplificada.

Ao final, crie uma classe de teste com o método `main` para instanciar objetos e demonstrar o funcionamento da classe, realizando operações entre frações e exibindo os resultados no formato fracionário e decimal.

2. Implemente uma classe `Conjunto` em Java, aplicando os conceitos de orientação a objetos estudados em sala. Sua classe deve representar um conjunto de números inteiros e possuir atributos privados, construtores adequados, métodos de acesso e métodos para realizar operações entre conjuntos.

A classe deve possuir, no mínimo:

- construtor padrão;
- construtor que receba um vetor de inteiros e a quantidade de elementos a serem inseridos no conjunto;
- método `uniao(Conjunto c1, Conjunto c2)` para armazenar no conjunto atual a união entre dois conjuntos;
- método `intersecao(Conjunto c1, Conjunto c2)` para armazenar no conjunto atual a interseção entre dois conjuntos;
- método `pertence(int valor)` para verificar se um elemento pertence ao conjunto;
- método `insere(int valor)` para inserir um elemento no conjunto;
- método `retira(int valor)` para remover um elemento do conjunto;
- método `vazio()` para verificar se o conjunto está vazio;
- método `contem(Conjunto c)` para verificar se um conjunto contém outro;
- método `estaContido(Conjunto c)` para verificar se o conjunto atual está contido em outro;
- método `igual(Conjunto c)` para verificar se dois conjuntos são iguais;
- método `imprime()` para exibir os elementos do conjunto.

Considere ainda as seguintes regras:

- os atributos da classe devem ser privados, garantindo o encapsulamento;
- o conjunto deve armazenar apenas números inteiros sem repetição;
- os métodos devem manipular corretamente os elementos do conjunto, respeitando a definição matemática de união, interseção, igualdade e subconjunto.

Ao final, crie uma classe de teste com o método `main` para instanciar objetos e demonstrar o funcionamento da classe, realizando operações entre conjuntos e exibindo os resultados.

3. Implemente, em Java, as classes `Circulo` e `Retangulo`, aplicando os conceitos de orientação a objetos estudados em sala. Cada classe deve possuir atributos privados, construtores adequados, métodos de acesso e um método para calcular a área da figura.

Atenção: nesta questão, não é para utilizar herança. As classes devem ser implementadas separadamente, cada uma com seus próprios atributos e métodos.

A classe `Circulo` deve possuir, no mínimo:

- atributos privados para armazenar a posição `x` e `y`, a `cor`, a `espessuraContorno`, o `tipoContorno` e o `raio`;
- construtor padrão;

- construtor parametrizado;
- métodos `set` e `get` para todos os atributos;
- método `imprime()` para exibir os dados do círculo;
- método `area()` para calcular e retornar a área do círculo.

A classe `Retangulo` deve possuir, no mínimo:

- atributos privados para armazenar a posição `x` e `y`, a `cor`, a `espessuraContorno`, o `tipoContorno`, a `largura` e a `altura`;
- construtor padrão;
- construtor parametrizado;
- métodos `set` e `get` para todos os atributos;
- método `imprime()` para exibir os dados do retângulo;
- método `area()` para calcular e retornar a área do retângulo.

Considere ainda as seguintes regras:

- os atributos devem ser privados, garantindo o encapsulamento;
- o valor da `cor` deve estar entre 1 e 5;
- o valor da `espessuraContorno` deve estar entre 1 e 5;
- o valor do `tipoContorno` deve estar entre 1 e 2;
- caso valores inválidos sejam informados, a classe deve atribuir valores padrão apropriados.

Ao final, crie uma classe de teste com o método `main` para instanciar objetos das duas classes, alterar alguns atributos com os métodos `set`, exibir seus dados com o método `imprime()` e mostrar o valor da área de cada figura.

4. Implemente, em Java, uma hierarquia de classes para representar figuras geométricas, aplicando os conceitos de orientação a objetos estudados em sala, com uso obrigatório de herança.

Crie uma classe base chamada `FormaBasica`, responsável por armazenar os atributos comuns das figuras. Essa classe deve possuir, no mínimo:

- atributos privados para armazenar a posição `x` e `y`, a `cor`, a `espessuraContorno` e o `tipoContorno`;
- construtor padrão;
- construtor parametrizado;
- métodos `set` e `get` para todos os atributos;
- método `imprime()` para exibir os dados básicos da figura.

Em seguida, crie a classe `Circulo`, que deve herdar de `FormaBasica` e possuir:

- atributo privado `raio`;

- construtor padrão;
- construtor parametrizado;
- métodos `setRaio()` e `getRaio()`;
- método `imprime()` para exibir os dados do círculo;
- método `area()` para calcular e retornar a área do círculo.

Depois, crie a classe `Retangulo`, que deve herdar de `FormaBasica` e possuir:

- atributos privados `largura` e `altura`;
- construtor padrão;
- construtor parametrizado;
- métodos `setLargura()`, `getLargura()`, `setAltura()` e `getAltura()`;
- método `imprime()` para exibir os dados do retângulo;
- método `area()` para calcular e retornar a área do retângulo.

Considere ainda as seguintes regras:

- os atributos devem ser privados, garantindo o encapsulamento;
- o valor da `cor` deve estar entre 1 e 5;
- o valor da `espessuraContorno` deve estar entre 1 e 5;
- o valor do `tipoContorno` deve estar entre 1 e 2;
- caso valores inválidos sejam informados, a classe deve atribuir valores padrão apropriados;
- nesta questão, é obrigatório utilizar herança para evitar repetição de código entre as classes.

Ao final, crie uma classe de teste com o método `main` para instanciar objetos das classes `Circulo` e `Retangulo`, alterar alguns atributos com os métodos `set`, exibir seus dados com o método `imprime()` e mostrar o valor da área de cada figura.